Describe Your Solution — form field answers

Paste each block into the matching field in the Security Review submission form. Anything in `[SQUARE BRACKETS]` must be replaced before submitting.

Field: "Provide a detailed technical description of your solution and related components."

Who Sees What is a read-only administrative tool that answers a single question for a Salesforce System Administrator: what can a given user see and do, and what granted it.

An admin selects a user, then drills from apps to objects to fields. For each object the tool reports create, read, update, delete, view-all and modify-all. For each field it reports readable and editable. Every grant is traced back to the permission set, permission set group, or profile that produced it.

The package is 100% native Salesforce Platform. It consists of:

- 16 Apex classes plus 6 Apex test classes, all declared `with sharing`
- 22 Lightning Web Component bundles
- 1 custom object, `PE_Inspection_Log__c`, with 5 custom fields
- 1 custom permission, `Use_Permission_Explorer`
- 1 permission set, `PermissionCheckerPermissionSet`
- 1 Lightning Component tab and 1 Lightning app
- 1 Platform Cache partition, `PermissionExplorer`

Second-generation managed package under namespace `twinforce`.

Authorisation. Every `@AuraEnabled` entry point on `PermissionExplorerController` begins by calling `PEAccessGate.assertAuthorised()`, which requires the packaged custom permission `Use_Permission_Explorer` via `FeatureManagement.checkPermission`. Without that permission every entry point throws before any query executes. This is asserted by `PermissionExplorerControllerTest.unauthorisedCallerIsRejectedByEveryEntryPoint` and `PEUserDetailServiceTest.unauthorisedUserIsRejectedBeforeAnyQuery`. The custom permission is granted only by the packaged permission set, which the subscriber's administrator must assign deliberately; the package does not auto-assign it.

Enforcement. All classes are `with sharing`. All SOQL executes in user mode — either `WITH USER_MODE` on inline queries or `Database.queryWithBinds(soql, binds, AccessLevel.USER_MODE)` for the one dynamically-assembled query — so object and field permissions are enforced for the running user by the platform, not by our code.

Objects read. The tool reads only permission metadata and user records. It writes only to its own audit object. Objects queried:

`User`, `UserLogin`, `UserRole`, `Profile`, `PermissionSet`, `PermissionSetAssignment`, `PermissionSetGroup`, `PermissionSetGroupComponent`, `PermissionSetTabSetting`, `ObjectPermissions`, `FieldPermissions`, `LoginHistory`, `AppMenuItem`, `TabDefinition`, `EntityDefinition`.

Auditing. Every inspection writes a row to `PE_Inspection_Log__c` recording the inspecting user, the inspected user, the action, and a UTC timestamp. The packaged permission set grants Read and Create on that object and explicitly withholds Edit and Delete, so holders can read the log but cannot alter or remove entries.

Deliberate limitations, disclosed in the UI. Per-user record type visibility and per-permission-set app assignment exist only inside Profile and PermissionSet metadata XML, which is reachable only via the Metadata or Tooling API — that is, via a callout. This package makes no callouts, so those two areas are reported as "not analysed" in the interface rather than guessed at. Login history requires the "Manage Users" permission, which the package does not grant; where the running admin lacks it the panel states that instead of showing an empty list.

Dynamic SOQL. One query is assembled at runtime, in `PEUserSearchService.search`, to apply optional filters. It is built exclusively from `private static final` string constants; no user-supplied value is ever concatenated into the string. All values are passed through a named bind map to `Database.queryWithBinds`. The only varying identifier is the sort column, which is validated against an allowlist (`PEKeysetCursor.isAllowedSort`) and then applied by selecting between two constant `ORDER BY` fragments rather than by interpolation. Tests `PEUserSearchServiceTest.quoteInTermIsBoundNotInterpolated` and `PermissionExplorerTest.disallowedSortFieldIsRejected` assert this.

Test suite: 101 Apex tests, 100% passing. Package code coverage 88%.

Field: "If your solution includes or requires a web app or service, list the frameworks and languages that it uses."

Set the toggle to DISABLED.

If the field must be completed: The package does not include or require any web application or web service. All logic is Apex and Lightning Web Components executing on the Salesforce Platform.

Field: "If your solution uses any platforms other than Salesforce Platform, list them and describe how they're used."

Set the toggle to DISABLED.

If the field must be completed: No platforms other than the Salesforce Platform are used. There are no Heroku, Marketing Cloud, or external hosting components.

Field: "If your solution integrates with technology outside the Salesforce Platform, provide details."

Set the toggle to DISABLED.

If the field must be completed: The package performs no integrations outside the Salesforce Platform. It contains no HTTP callouts, no Named Credentials, no External Credentials, and no Remote Site Settings. This can be verified by searching the package source for `HttpRequest`, `new Http()`, and `callout:` — all return zero matches.

Field: "If your solution contains Salesforce Platform technology, such as Lightning Components and Apex, provide details."

Toggle ENABLED.

Apex: 16 classes plus 6 test classes, all `public with sharing`. Entry points are the `@AuraEnabled` methods on `PermissionExplorerController` (`searchUsers`, `resolveUser`, `getAppObjects`, `getObjectDetail`, `getFilterOptions`, `getLoginHistory`, `getAssignedPermissionSets`, `getAssignedPermissionSetGroups`, `refreshCache`). Every one calls `PEAccessGate.assertAuthorised()` first. All SOQL runs in user mode via `WITH USER_MODE` or `AccessLevel.USER_MODE`.

Lightning Web Components: 22 bundles. `permissionExplorer` is the top-level component, exposed as a Lightning Component tab. All others are internal children with `isExposed` false. Components hold no credentials and make no network calls of their own; all server communication is through the `@AuraEnabled` Apex methods listed above.

Custom object: `PE_Inspection_Log__c` — append-only audit trail, five fields (Action, Detail, Inspected At, Inspector User, Subject User).

Custom permission: `Use_Permission_Explorer` — the sole authorisation gate.

Platform Cache: partition `PermissionExplorer`, used to cache resolved permission results for 15 minutes (session scope) and describe metadata for 1 hour (org scope). Cache absence is handled gracefully: `PECacheManager` falls back to a transaction-scoped map, so the package functions correctly in subscriber orgs with no cache allocation. No sensitive data beyond permission metadata already visible to the running admin is cached.

Visualforce: none. Triggers: none. Flows: none. Aura components: none.

Field: “If your solution stores any Salesforce OAuth tokens or other access tokens, describe how the tokens are stored.”

Set the toggle to DISABLED.

If the field must be completed: The package stores no OAuth tokens, session tokens, API keys, passwords, or credentials of any kind. There is no authentication flow; the package operates entirely within the subscriber’s authenticated Salesforce session. It contains no Named Credentials, External Credentials, Auth Providers, or Connected Apps, and no custom setting or custom object field holds credential data.

Field: “If your solution stores any Salesforce data outside the Salesforce Platform, list all types of stored data, and describe where and how the data is stored.”

Set the toggle to DISABLED.

If the field must be completed: No Salesforce data is stored outside the Salesforce Platform. The package transmits nothing off-platform. All data written by the package resides in the subscriber’s own org, in the `PE_Inspection_Log__c` custom object and in the subscriber’s Platform Cache partition.